

שאלה 1 (40 נקודות) - על שאלה זו ענו במחברת. קראו את כל השאלה לפני שתתחילו לענות.

סעיף א

כתבו מחלקה גנרית בשם Repository, המייצגת מאגר, שמקבלת שני טיפוסים גנריים: ITEM ו-CREATOR. הטיפוס הגנרי ITEM מייצג מחלקה שמייצגת פריט, והטיפוס הגנרי CREATOR מייצג מחלקה שמייצגת את היוצר שלו. על המחלקה להכיל אוסף מסוג Map שישמור עבור כל יוצר אוסף של כל הפריטים שיצר. כלומר, כל הפריטים שנוצרו על ידי אותו יוצר יישמרו באוסף אחד, והגישה אליו תתבצע באמצעות Map. אי אפשר לשמור עבור יוצר אחד כמה פריטים זהים, אם בכל זאת נשמור, רק פריט אחד ישמר.

כיתבו למחלקה את המתודות הבאות:

- add – מקבלת פריט ויוצר ומוסיפה את הפריט למאגר.
- getItemsByCreator – מקבלת יוצר ומחזירה אוסף של כל הפריטים שיצר שבמאגר ממוינים בהתאם לדרך ההשוואה שמגדירה המחלקה ITEM בין העצמים שלה.
- getCreatorsOfItem – מקבלת פריט ומחזירה אוסף של כל היוצרים שיצרו פריט כזה ממוינים לפי סדר ההכנסה שלהם (יכולים להיות כמה יוצרים שיצרו את אותו הפריט).

בנוסף על המחלקה לממש את הממשק Iterable שמחזיר עבור כל יוצר מחרוזת שמורכבת ממחרוזת שמייצגת את היוצר (toString) ומחרוזת שמייצגת את הפריטים שאותם יצר (toString) בהתאם לדוגמה שבהמשך. סדר היוצרים שהאיטרטור יחזיר צריך להיות בהתאם לסדר ההכנסה של היוצרים לאוסף. כלומר שיוצר שנכנס למאגר ראשון (כלומר שאחד מפריטיו נכנס למאגר ראשון) יוחזר ראשון. סדר הפריטים שהאיטרטור מחזיר עבור כל יוצר במחרוזת צריך להיות ממין בהתאם לדרך ההשוואה שמגדירה המחלקה ITEM בין העצמים שלה.

סעיף ב

בהינתן המימוש של המחלקה Book והמחלקה Author. כיתבו מה נדרש להשלים במימוש כך שהמחלקות יוכלו לשמש במחלקה Repository כטיפוסים ITEM ו-CREATOR (בדומה לדוגמה בהמשך).

```
class Book {
    private String name;
    public Book(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public String toString() {
        return name;
    }
}
```

```

class Author {
    private String firstName;
    private String lastName;
    public Author(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
    public String getFirstName() {
        return firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public String toString() {
        return firstName + " " + lastName;
    }
}

```

דוגמה לשימוש במחלקה Repository ובמחלקות Author ו-Book והדפסות שמתקבלות

```
Repository<Book, Author> library = new Repository<>();
```

```

Author author1 = new Author("George", "Orwell");
Author author2 = new Author("Jane", "Austen");
library.add(new Book("Ccc"), author1);
library.add(new Book("Aaa"), author1);
library.add(new Book("Baa"), author1);
library.add(new Book("Ddd"), author2);
library.add(new Book("Aaa"), author2);
for (String s : library) {
    System.out.println(s);
}

```

ידפיס:

```

Creator: George Orwell, Items: Aaa Baa Ccc
Creator: Jane Austen, Items: Aaa Ddd

```

```

System.out.println(
    library.getCreatorsOfItem(new Book("Aaa")));

```

ידפיס:

```
[George Orwell, Jane Austen]
```

```
System.out.println(library.getItemsByCreator(author1));
```

ידפיס:

```
[Aaa, Baa, Ccc]
```

```

class Repository<ITEM extends Comparable<ITEM>, CREATOR> implements
Iterable<String> {
    private Map<CREATOR, Collection<ITEM>> map =
        new LinkedHashMap<>();

    public void add(ITEM item, CREATOR creator) {
        if(!map.containsKey(creator))
            map.put(creator, new TreeSet<>());
        map.get(creator).add(item);
    }
    public Collection<ITEM> getItemsByCreator(CREATOR creator) {
        return map.get(creator);
    }
    public Collection<CREATOR> getCreatorsOfItem(ITEM item) {
        Set<CREATOR> creatorsWithItem =
            new LinkedHashSet<>();
        for (CREATOR creator : map.keySet()) {
            if (map.get(creator).contains(item))
                creatorsWithItem.add(creator);
        }
        return creatorsWithItem;
    }

    @Override
    public Iterator<String> iterator() {
        return new Iterator<String>() {
            Iterator<CREATOR> creatorIter =
                map.keySet().iterator();

            @Override
            public boolean hasNext() {
                return creatorIter.hasNext();
            }
            @Override
            public String next() {
                CREATOR curCreator = creatorIter.next();
                Collection<ITEM> itemsForCurCreator =
                    map.get(curCreator);
                StringBuilder sb = new StringBuilder();
                sb.append("Creator: "+curCreator+", Items: ");
                for(ITEM item: itemsForCurCreator)
                    sb.append(item + " ");
                return sb.toString();
            }
        };
    }
}

```

סעיף ב

התוספת הדרושה ל-Book:

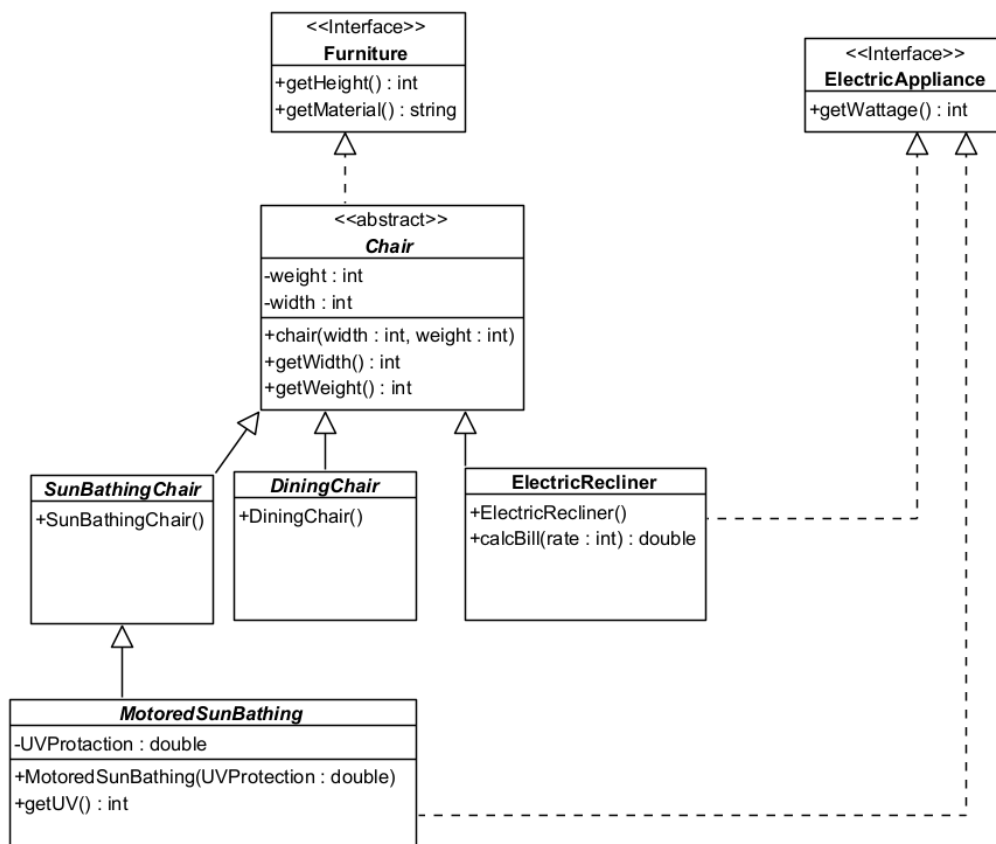
```
@Override
public boolean equals(Object obj) {
    Book other = (Book) obj;
    return Objects.equals(name, other.name);
}
@Override
public int compareTo(Book o) {
    return this.name.compareTo(o.name);
}
```

התוספת הדרושה ל-Author:

```
@Override
public int hashCode() {
    return Objects.hash(firstName, lastName);
}
@Override
public boolean equals(Object obj) {
    Author other = (Author) obj;
    return Objects.equals(firstName, other.firstName) &&
        Objects.equals(lastName, other.lastName);
}
```

שאלה 2 (40 נקודות) - על שאלה זו ענו בטופס

בשאלה זו 10 סעיפים, כל סעיף שווה 4 נקודות. לכל סעיף יש לסמן את התשובה הנכונה



סעיפים 1-3 מתייחסים לקוד הבא, שמתקמפל ורץ בצורה תקינה:

```

_____(1)_____ x = new MotoredSunBathing(25);

_____(2)_____ y = new DiningChair();

_____(3)_____ z = new ElectricRecliner();

System.out.println(x.getHeight());

System.out.println(y.getWeight());

y=x;

System.out.println(y.getWidth());

z=x;

System.out.println(z.getWeight());

System.out.println(x.getWattage());

System.out.println(new ElectricRecliner().calcBill(10));

```

1. במיקום (1) צריך להיות?

2. במיקום (2) צריך להיות?

3. במיקום (3) צריך להיות?

4. להלן המימוש של ElectricRecliner() שמופיע בתרשים:

```
public ElectricRecliner() {  
  
    System.out.println("Object Generated");  
  
}
```

5. רוצים להוסיף את המתודה הבאה לממשק (ממשק) ElectricAppliance (בנוסף לקיים בתרשים):

```
public String getMaterial();
```

6. רוצים להוסיף את המתודה הבאה למחלקה MotoredSunBathing (בנוסף לקיים בתרשים):

```
public double getWattage(){  
    return Math.PI;  
}
```

7. בהינתן הקוד הבא במתודת main:

```
Chair c1 = new ElectricRecliner();  
Chair c2 = new MotoredSunBathing(10);  
System.out.println(c1);  
c2=c1;  
System.out.println(((ElectricAppliance)c2).getWattage());
```

8. בהינתן הקוד הבא במתודת main:

```
Chair c3 = new ElectricRecliner();  
Chair c4 = new SunBathingChair();  
System.out.println(c3);  
c3=c4;  
System.out.println(((ElectricAppliance)c3).getWattage());
```

9. רוצים להוסיף את המתודה הבאה למחלקה ElectricRecliner (בנוסף לקיים בתרשים):

```
public double getMaxReclineHeight() {  
    return 0.5*super.width;  
}
```

10. רוצים להוסיף את המתודה הבאה למחלקה ElectricRecliner (בנוסף לקיים בתרשים):

```
public double getWidth(int a) {  
    return 0.5*(a%3);  
}
```

שאלה 2 תשובות

#	א	ב	ג	ד
1	Furniture	Chair	sunBathingChair	MotoredSunBathing
2	Furniture	Chair	DiningChair	sunBathingChair
3	Furniture	Chair	ElectricRecliner	ElectricAppliance
4	הקוד שגוי ויגרום לשגיאת קומפילציה	הקוד תקין ומהווה דריסה פולימורפית	הקוד תקין ומהווה העמסה	שגיאת זמן ריצה
5	הקוד תקין וניתן למימוש	הקוד תקין ומהווה דריסה של ממשק Furniture	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה
6	הקוד תקין ומהווה העמסה	הקוד תקין ומהווה דריסה	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה
7	הקוד תקין	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל ההשמה של c1 ל c2.	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל הקריאה למתודה getWattage
8	הקוד תקין	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל ההשמה של c4 ל c3.	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל הקריאה למתודה getWattage
9	הקוד תקין ומהווה העמסה	הקוד תקין ומהווה דריסה	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה
10	הקוד תקין ומהווה העמסה	הקוד תקין ומהווה דריסה	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה

פיתרון שאלה 2:

#	א	ב	ג	ד
1	Furniture	Chair	sunBathingChair	MotoredSunBathing
2	Furniture	Chair	DiningChair	sunBathingChair
3	Furniture	Chair	ElectricRecliner	ElectricAppliance
4	הקוד שגוי ויגרום לשגיאת קומפילציה	הקוד תקין ומהווה דריסה פולימורפית	הקוד תקין ומהווה העמסה	שגיאת זמן ריצה
5	הקוד תקין וניתן למימוש	הקוד תקין ומהווה דריסה של ממשק Furniture	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה
6	הקוד תקין ומהווה העמסה	הקוד תקין ומהווה דריסה	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה
7	הקוד תקין	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל ההשמה של c1 ל c2.	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל הקריאה למתודה getWattage
8	הקוד תקין	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל ההשמה של c3 ל c4.	הקוד שגוי ויגרום לשגיאת קומפילציה בגלל הקריאה למתודה getWattage
9	הקוד תקין ומהווה העמסה	הקוד תקין ומהווה דריסה	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה
10	הקוד תקין ומהווה העמסה	הקוד תקין ומהווה דריסה	הקוד שגוי ויגרום לשגיאת זמן ריצה	הקוד שגוי ויגרום לשגיאת קומפילציה

1. ד

2. ב

3. ב

4. א – כי אין בנאי ריק אצל מחלקת Chair

5. א

6. ד יש כבר מתודה בחתימה זהה המחזירה טיפוס משתנה אחר.

7. א

8. ב – נסיון להמיר ל type לא מתאים

9. ד - יש ניסיון לשימוש מחוץ למחלקה בשדה שהוא פרטי

10. א

שאלה 3 (20 נקודות) – על שאלה זו ענו בטופס

השאלה מורכבת משתי תת שאלות שונות.

שאלה 3א:

נתון הקוד הבא להרצה של קוביות משחק:

```
public class Cubes implements Runnable {
    private int count;
    private long total;
    private int max;

    Cubes(int max) {
        this.max = max;
    }

    private void inc() {
        count = (count + 1) % max;
        total++;
    }

    public int getCount() {
        return count;
    }

    public long getTotal() {
        return total;
    }

    public void startAgain() {
        count = 0;
        total = 0;
    }

    @Override
    public void run() {
        while (true) {
            try {
                Thread.sleep(1);
            } catch (InterruptedException e) {
                return;
            }
            inc();
        }
    }
}
```

1. בעת ההרצה במקביל של מספר חוטים המשתמשים במחלקה זו עלולה להיות בעיה בקריאה למתודה startAgain. מה הבעיה וכיצד ניתן לפתור אותה?

2. יש לבנות פונקציה main באופן הבא:

a. הפונקציה תריץ שתי קוביות במקביל.

b. כאשר המשתמש ילחץ על מעבר שורה במקלדת (Enter) התוכנית תקרא את הערך של כל אחת מהקוביות ותדפיסו.

c. התוכנית תמתין שניה, תפסיק את פעולת הקוביות ותחכה עד שהחוטים של שתי הקוביות יסיימו ואז תדפיס את הערך total של כל אחת מהקוביות.

תשובה:

פתרון שאלה 3א

1. הבעיה היא שיכול להיות מצב שהמתודה startAgain תקרא מחוט אחר באופן שה-context switch בין החוט של הקוביה לחוט שקרא ל-startAgain מתרחש באמצע הקריאה למתודה inc, אחרי שהתבצעה הקריאה של השדה count או final ולפני עידכונו. במצב כזה אחרי סיום המתודה startAgain שמאתחלת את המשתנים המתודה inc תחזור לפעולה ותעדכן את השדות count ו-final ותדרוס את האיתחול. הפתרון הוא להוסיף את המאפיין synchronize לכל אחת מהפונקציות.

2.

```
public static void main(String[] args) {
    Cubes cube1 = new Cubes(6);
    Cubes cube2 = new Cubes(6);
    Thread t1 = new Thread(cube1);
    Thread t2 = new Thread(cube2);
    t1.start();
    t2.start();
    Scanner scanner = new Scanner(System.in);
    scanner.nextLine();
    int cube1Val = cube1.getCount();
    int cube2Val = cube2.getCount();
    System.out.println(cube1Val+" "+cube2Val);
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    t1.interrupt();
    t2.interrupt();
    try {
        t1.join();
        t2.join();
    } catch (InterruptedException e) {
    }
    System.out.println("Total: " + cube1.getTotal() +
        " " + cube2.getTotal());

    scanner.close();
}
```

שאלה 3

מה תדפיס התוכנית הבאה ?

```
class ExceptionA extends Exception {
    public ExceptionA() {System.out.print(" exA");}
}
class ExceptionB extends Exception {
    public ExceptionB() {System.out.print(" exB");}
}
public class Thrower {
    public static void a() throws Exception {
        try {
            b();
            System.out.print(" a1");
        }
        catch (ExceptionB e) {
            System.out.print(" a2");
            throw new ExceptionA();
        }
        finally{
            System.out.print(" fina");
        }
        System.out.print(" Enda");
    }
    public static void b() throws Exception {
        try {
            System.out.print(" b1");
            throw new ExceptionB();
        }
        catch (Exception e) {
            System.out.print(" b2");
            throw new ExceptionA();
        }
    }

    public static void main(String[] args) {
        try {
            a();
        }
        catch (Exception e) {
            System.out.println(" catch!");
        }
    }
}
```

תשובה:

פתרון: b1 exB b2 exA fina catch!